Diffusion Language Models

Phạm Thanh Hưng

I. Dẫn nhập: Vì sao phải sinh văn bản từ trái sang phải?

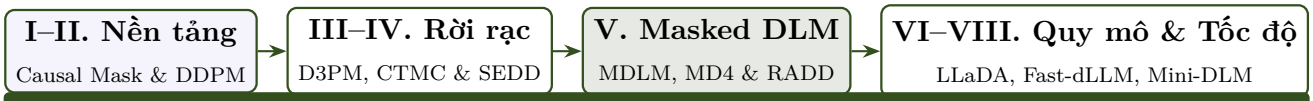
Trong hơn nửa thập kỷ qua, trí tuệ nhân tạo tạo sinh (*Generative AI*) được dẫn dắt bởi các mô hình ngôn ngữ lớn tự hồi quy (*Autoregressive Large Language Models* – AR LLMs) [1], vận hành trên quy tắc nhân xích xác suất:

$$p_\theta(x) = \prod_{i=1}^{L} p_\theta(x_i \mid x_{<i}) = \prod_{i=1}^{L} p_\theta(x_i \mid x_1, \ldots, x_{i-1}) \tag{1}$$

Trong đó $x = (x_1, \ldots, x_L)$ là chuỗi L token thuộc từ vựng $\mathcal{V}$, $x_{<i}$ là tiền tố đã sinh trước bước i, và $p_\theta(x_i \mid x_{<i})$ là phân phối xác suất có điều kiện do mạng Transformer tham số hóa bởi θ dự đoán.

Mặc dù mặt nạ nhân quả (*causal mask*) hỗ trợ huấn luyện song song hiệu quả, cơ chế sinh tuần tự từ trái sang phải áp đặt ba điểm nghẽn cố hữu: (1) độ trễ suy luận tuyến tính $\mathcal{O}(L)$ bị nghẽn băng thông bộ nhớ (*Memory-Bound*); (2) bất lực trong tác vụ điền vào giữa (*infilling*) và chỉnh sửa cục bộ do thiếu ngữ cảnh tương lai $x_{>i}$; và (3) **Lời nguyền đảo ngược** (*The Reversal Curse*) khiến mô hình học quan hệ xuôi $A \to B$ nhưng thất bại khi suy luận ngược $B \to A$ [2].

Mô thức mới – Diffusion Language Models (DLMs): Khởi đầu từ chuỗi toàn mặt nạ $\texttt{[M]}^L$, DLM áp dụng tự chú ý hai chiều toàn cục (*Full Bidirectional Attention*) để khử nhiễu song song nhiều vị trí qua S bước tinh chỉnh ($S \ll L$), giải phóng mô hình khỏi ràng buộc nhân quả tuần tự và khắc phục tự nhiên Lời nguyền đảo ngược.



Hình 1: Lộ trình chuyển dịch từ mô hình tự hồi quy sang mô hình ngôn ngữ khuếch tán.

I.1. Bức tranh toàn cảnh các mô hình Diffusion Language Models hiện nay

Hệ sinh thái DLM hiện đại (2021–2026) định hình qua ba làn sóng kiến trúc chủ đạo: (1) **Nền tảng lý thuyết rời rạc:** D3PM [3] (ma trận Q_t), SEDD [4] (Concrete Score liên tục), MDLM [5] và MD4 [6] (loss cross-entropy mặt nạ trọng số $1/t$); (2) **Mô hình nền tảng quy mô lớn (*Frontier DLMs*):** LLaDA-8B [2] (huấn luyện từ đầu trên 2,3T token) và Dream-7B [7] (chuyển giao trọng số từ Qwen2.5-7B, tiết kiệm 95% chi phí pre-training); (3) **Tối ưu chuyên biệt & tốc độ cao:** DiffuCoder-7B [8] (mã nguồn), Block Diffusion [9], Fast-dLLM [10] (lưu đệm KV xấp xỉ) và Mercury Coder [11] ($> 1\,000$ token/s trên H100).

Tóm lại, việc từ bỏ cơ chế sinh một chiều là bước chuyển dịch tất yếu nhằm giải phóng mô hình ngôn ngữ khỏi các điểm nghẽn cố hữu. Phần II. sẽ khảo sát nền tảng toán học kết nối giữa mô hình tự hồi quy và quá trình khuếch tán.

Mục lục

I. Dẫn nhập: Vì sao phải sinh văn bản từ trái sang phải? . . . 1
I.1. Bức tranh toàn cảnh các mô hình Diffusion Language Models hiện nay . . . 1
II. Nền tảng: Từ mô hình tự hồi quy đến mô hình khuếch tán . . . 3
II.1. Mô hình tự hồi quy và cơ chế mặt nạ nhân quả . . . 3
II.2. Khuếch tán trên dữ liệu liên tục: Trực giác từ DDPM . . . 4
II.3. Vì sao văn bản khó: Thất bại của khuếch tán Gauss trên không gian rời rạc . . . 4
II.4. Cầu nối sang không gian rời rạc: Xích Markov và ma trận chuyển . . . 5
III. Khuếch tán rời rạc: Mô hình D3PM và ma trận chuyển trạng thái . . . 5
III.1. Cấu trúc toán học của mô hình D3PM . . . 6
III.2. Ba dạng cấu trúc ma trận chuyển trạng thái . . . 6
III.3. Phân phối hậu nghiệm ngược và Cận dưới biến phân (ELBO) . . . 7
III.4. Ví dụ tính tay D3PM hoàn chỉnh từng bước . . . 7
IV. Score-based Discrete Diffusion: Tiếp cận thời gian liên tục với SEDD . . . 8
IV.1. Xích Markov thời gian liên tục và ma trận tốc độ chuyển trạng thái . . . 9
IV.2. Khái niệm Concrete Score và Hàm mục tiêu Score Entropy . . . 9
IV.3. Ví dụ tính tay: Tính toán Concrete Score và Score Entropy Loss . . . 10
V. Masked Discrete Diffusion: Cơ chế che dần và tối ưu hóa MDLM/MD4 . . . 10
V.1. Quá trình khuếch tán xuôi hấp thụ và Tỷ lệ rủi ro (Hazard Rate) . . . 11
V.2. Đột phá lý thuyết: Đơn giản hóa ELBO thành Weighted Masked Cross-Entropy . 11
V.3. Cơ chế Attention: Causal Attention vs Full Bidirectional Attention . . . 12
V.4. Ví dụ tính tay: Tính toán hàm mất mát MDLM/MD4 trên câu 5 token . . . 12
VI. Mở rộng quy mô và thuật toán lấy mẫu: LLaDA, Dream và Remasking . . . 13
VI.1. Các mô hình nền tảng khuếch tán quy mô lớn (Frontier DLMs) . . . 13
VI.2. Thuật toán lấy mẫu và Classifier-Free Guidance trong miền rời rạc . . . 14
VI.3. Chiến lược Che lại theo Độ tin cậy (Low-Confidence Remasking) . . . 15
VI.4. Khuếch tán theo Khối (Block Diffusion - BD3-LM) . . . 15
VII. Tăng tốc suy luận và kiến trúc hệ thống: Thách thức KV-Cache và Fast-dLLM 16
VII.1. Nghịch lý KV Cache trong cơ chế chú ý hai chiều . . . 16
VII.2. Các giải pháp xấp xỉ kiến trúc hệ thống và Fast-dLLM . . . 17
VII.3. Bằng chứng toán học: Thất bại của giả định độc lập có điều kiện . . . 18
VII.4. Đánh đổi không gian–thời gian và hệ sinh thái phục vụ công nghiệp . . . 19
VIII. Thực hành mô phỏng: Xây dựng và huấn luyện Mini-DLM với PyTorch . . . 19
VIII.1. Thiết kế kiến trúc mô hình và ngữ liệu sư phạm . . . 19
VIII.2. Quy trình huấn luyện và tối ưu hóa hàm mất mát . . . 20
VIII.3. Hành trình khử mặt nạ từng bước và Điền khuyết hai đầu . . . 21
IX. So sánh tổng hợp AR và DLM: Đánh giá toàn diện và triển vọng tương lai . 23
IX.1. Bảng ma trận so sánh đối đầu 10 tiêu chí kỹ thuật . . . 23
IX.2. Động học KV Cache và Những giới hạn thẳng thắn của DLM . . . 24
IX.3. Tổng hợp tiến trình phát triển và Triển vọng nghiên cứu tương lai . . . 25
Tài liệu tham khảo . . . 26
Phụ lục . . . 28
Tài nguyên mã nguồn mở và thực nghiệm . . . 28

II. Nền tảng: Từ mô hình tự hồi quy đến mô hình khuếch tán

Để xây dựng nền tảng trực giác vững chắc về mô hình ngôn ngữ khuếch tán, trước tiên ta cần phân tích sâu sắc cơ chế hoạt động của mô hình tự hồi quy truyền thống và đối chiếu với nguyên lý khử nhiễu liên tục trong các mô hình khuếch tán xác suất (*Denoising Diffusion Probabilistic Models* – DDPM) [12, 13]. Sự đối lập giữa tính phụ thuộc nhân quả một chiều và quá trình tái tạo toàn thể hai chiều chính là chìa khóa mở ra tư duy thiết kế cho các mô hình khuếch tán ngôn ngữ.

II.1. Mô hình tự hồi quy và cơ chế mặt nạ nhân quả

Mô hình tự hồi quy mô hình hóa phân phối xác suất đồng thời của chuỗi $x = (x_1, \ldots, x_L)$ thông qua quy tắc nhân xác suất có điều kiện:

$$p_\theta(x) = \prod_{i=1}^{L} p_\theta(x_i \mid x_{<i}) \tag{2}$$

Trong đó $x = (x_1, \ldots, x_L)$ là chuỗi L token thuộc từ vựng $\mathcal{V}$, x_i là token cần dự đoán tại vị trí i, $x_{<i} = (x_1, \ldots, x_{i-1})$ là tiền tố đã sinh, và $p_\theta(x_i \mid x_{<i})$ là phân phối xác suất danh mục do mạng Transformer tham số hóa.

Trong kiến trúc Transformer giải mã [1], tính phụ thuộc nhân quả được thực thi thông qua cơ chế tự chú ý có mặt nạ:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M_{\text{causal}}\right)V \tag{3}$$

Trong đó $Q, K, V \in \mathbb{R}^{L \times d_k}$ là ma trận Query, Key, Value; $\sqrt{d_k}$ là hệ số tỷ lệ chuẩn hóa; và $M_{\text{causal}} \in \{0, -\infty\}^{L \times L}$ là mặt nạ nhân quả tam giác dưới với $[M_{\text{causal}}]_{ij} = 0$ khi $j \leq i$ và $-\infty$ khi $j > i$.

Ví dụ tính tay – Tính xác suất chuỗi AR: Xét chuỗi 3 token $x =$ (“Học”, “sâu”, “ứng”). Các xác suất có điều kiện giả định được trình bày trong Bảng 1.

Bước (i)	**Ngữ cảnh** ($x_{<i}$)	**Token** (x_i)	**Xác suất có điều kiện** $p_\theta(x_i \mid x_{<i})$
1	$\emptyset$	“Học”	$p(x_1 =$ “Học”$) = 0{,}4000$
2	(“Học”)	“sâu”	$p(x_2 =$ “sâu” $\mid x_1) = 0{,}2500$
3	(“Học”, “sâu”)	“ứng”	$p(x_3 =$ “ứng” $\mid x_1, x_2) = 0{,}8000$

Bảng 1: Bảng tính tay xác suất có điều kiện từng bước cho chuỗi 3 token “Học sâu ứng”.

Áp dụng quy tắc nhân xích (2), xác suất đồng thời của chuỗi là:

$$p_\theta(x) = 0{,}4000 \times 0{,}2500 \times 0{,}8000 = 0{,}0800$$

Các số hạng log-likelihood lần lượt là $\ln(0{,}4000) \approx -0{,}9163$, $\ln(0{,}2500) \approx -1{,}3863$, $\ln(0{,}8000) \approx -0{,}2231$. Tổng log-likelihood trên toàn chuỗi là $\sum_{i=1}^{3} \ln p_\theta(x_i \mid x_{<i}) = -2{,}5257$. Hàm mất mát negative log-likelihood trung bình là $\mathcal{L}_{\text{NLL}} = -\frac{1}{3}(-2{,}5257) \approx 0{,}8419$, tương ứng độ hỗn loạn $\text{PPL} = \exp(0{,}8419) \approx 2{,}3208$. Mọi bước sinh đều phụ thuộc tuần tự, không thể sửa sai nếu bước trước bị lệch.

II.2. Khuếch tán trên dữ liệu liên tục: Trực giác từ DDPM

Mô hình khuếch tán xác suất (*DDPM*) [13] tiếp cận bài toán sinh dữ liệu như quá trình đảo ngược thời gian của một xích thêm nhiễu ngẫu nhiên. Với dữ liệu liên tục $x_0 \in \mathbb{R}^d$, quá trình xuôi bổ sung nhiễu Gaussian qua T bước:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\, \sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\right) \tag{4}$$

Trong đó $x_t \in \mathbb{R}^d$ là vector tại bước t, $\beta_t \in (0,1)$ là phương sai nhiễu, và I là ma trận đơn vị. Nhờ tính chất của phân phối Gauss, phân phối biên tại bước t có dạng đóng:

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t;\, \sqrt{\bar{\alpha}_t}\, x_0,\, (1-\bar{\alpha}_t) I\right) \tag{5}$$

Trong đó $\alpha_t = 1 - \beta_t$ và $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ là hệ số suy giảm tín hiệu tích lũy. Khi $T \to \infty$, $x_T \sim \mathcal{N}(0, I)$. Quá trình khử nhiễu ngược $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ được tối ưu bằng cận dưới biến phân ELBO qua độ phân kỳ KL.

II.3. Vì sao văn bản khó: Thất bại của khuếch tán Gauss trên không gian rời rạc

Khác với điểm ảnh liên tục, văn bản là dữ liệu rời rạc danh mục (*categorical discrete data*) với K token ($K = 32\,000$ trong LLaMA, $K = 50\,257$ trong GPT-2). Giữa các token không tồn tại cấu trúc tô-pô Euclid tự nhiên: việc cộng nhiễu $\epsilon \sim \mathcal{N}(0,1)$ vào chỉ số từ là vô nghĩa. Để giải quyết, cộng đồng nghiên cứu đã khám phá hai hướng chính:

1. Khuếch tán trên không gian nhúng liên tục (Diffusion-LM [14]): Nhúng token vào vector $\mathbb{R}^d$, thực hiện DDPM liên tục, và làm tròn ở bước cuối:

$$\hat{w} = \arg\min_{w' \in \mathcal{V}} \|\hat{x}_0 - e(w')\|_2 \tag{6}$$

Trong đó $\hat{w}$ là token được chọn, $\hat{x}_0$ là vector khử nhiễu cuối cùng, và $e(w')$ là vector nhúng của token w'. Tuy nhiên, phương pháp này gặp lỗi làm tròn (*rounding error*) do vector rơi vào khoảng trống giữa các từ, và hiện tượng sụp đổ biểu diễn (*representation collapse*).

2. Khuếch tán trực tiếp trên không gian rời rạc (D3PM [3]): Định nghĩa trực tiếp quá trình khuếch tán trên tập danh mục $\mathcal{V}$ qua ma trận chuyển Markov, loại bỏ hoàn toàn bước làm tròn.

Tiêu chí so sánh	Embedding Diffusion (Diffusion-LM)	Khuếch tán Rời rạc (D3PM, MDLM)
Không gian hoạt động	Vector liên tục $\mathbb{R}^d$	Tập danh mục rời rạc $\mathcal{V} = \{1, \ldots, K\}$
Bản chất nhiễu	Gaussian liên tục $\mathcal{N}(0, \sigma^2 I)$	Ma trận chuyển Markov $Q_t \in \mathbb{R}^{K \times K}$
Bước làm tròn (Rounding)	Bắt buộc (chiếu láng giềng gần nhất)	**Không cần làm tròn**; sinh phân phối Categorical
Độ ổn định huấn luyện	Kém; nhạy cảm khởi tạo	Ổn định; tối ưu trực tiếp bằng cross-entropy và KL
Khả năng mở rộng	Khó vượt vài trăm triệu tham số	Mở rộng Foundation Model 8B (LLaDA, Dream)

Bảng 2: So sánh khuếch tán embedding liên tục và khuếch tán rời rạc.

II.4. Cầu nối sang không gian rời rạc: Xích Markov và ma trận chuyển

Trong không gian rời rạc K trạng thái, công cụ toán học tương đương là **xích Markov rời rạc theo thời gian**. Biểu diễn token dưới dạng vector one-hot hàng $\mathbf{e}_k \in \{0,1\}^K$, quá trình xuôi một bước được định nghĩa:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}(x_t;\, \mathbf{e}_{x_{t-1}} Q_t) \tag{7}$$

Trong đó $\mathrm{Cat}(x;\pi)$ là phân phối Categorical với vector xác suất $\pi \in [0,1]^K$, và $Q_t \in \mathbb{R}^{K \times K}$ là ma trận chuyển trạng thái ngẫu nhiên với $[Q_t]_{jk} = q(x_t = k \mid x_{t-1} = j)$ thỏa mãn tổng mỗi hàng bằng 1: $\sum_{k=1}^{K} [Q_t]_{jk} = 1$.

Tùy theo cấu trúc của Q_t, ta có hai dạng nhiễu chính: (1) **Nhiễu đều (Uniform Noise):** token bị đột biến ngẫu nhiên sang bất kỳ từ nào; (2) **Trạng thái hấp thụ (Absorbing State / Mặt nạ [M]):** token bị che dần bởi [M] và một khi đã bị che thì không thể thay đổi.

Tóm lại, các giới hạn của mô hình tự hồi quy và sự không tương thích của nhiễu Gaussian liên tục trên văn bản đã định hình yêu cầu cấp thiết về một khuôn khổ khuếch tán trực tiếp trên xích Markov rời rạc. Phần III. sẽ đi sâu vào mô hình D3PM, thiết lập công thức nghiệm đóng cho ma trận chuyển tích lũy, phân phối hậu nghiệm đảo ngược và hàm mất mát biến phân ELBO.

III. Khuếch tán rời rạc: Mô hình D3PM và ma trận chuyển trạng thái

Công trình D3PM (*Structured Denoising Diffusion Models in Discrete State-Spaces*) của Austin et al. (NeurIPS 2021) [3] đặt nền móng toán học đầu tiên cho khuếch tán trên không gian rời rạc bằng cách thay nhiễu Gaussian bằng các bước nhảy xích Markov có cấu trúc, cho phép tính

nghiệm đóng phân phối biên và tối ưu cận dưới biến phân ELBO chuẩn xác.

III.1. Cấu trúc toán học của mô hình D3PM

Xét từ vựng K trạng thái rời rạc $\mathcal{V} = \{1, \dots, K\}$, trong đó mỗi token được biểu diễn bởi vector one-hot hàng $\mathbf{e}_k \in \{0,1\}^K$. Quá trình khuếch tán xuôi một bước và phân phối biên tích lũy sau t bước có dạng đóng:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}(x_t;\, \mathbf{e}_{x_{t-1}} Q_t), \qquad q(x_t \mid x_0) = \mathrm{Cat}(x_t;\, \mathbf{e}_{x_0} \bar{Q}_t), \qquad \bar{Q}_t = \prod_{s=1}^{t} Q_s \tag{8}$$

Trong đó:

- $x_t, x_{t-1}, x_0 \in \mathcal{V}$ – trạng thái token rời rạc tại bước t, $t-1$ và trạng thái sạch ban đầu ($t = 0$).
- $Q_t \in \mathbb{R}^{K \times K}$ – ma trận chuyển bước t ($\sum_{k=1}^{K} [Q_t]_{jk} = 1$); $\bar{Q}_t = Q_1 Q_2 \dots Q_t$ – ma trận chuyển tích lũy.
- $\mathrm{Cat}(x; \pi)$ – phân phối Categorical; $\mathbf{e}_{x_0} \bar{Q}_t$ chính là hàng thứ x_0 của $\bar{Q}_t$, cho phép lấy mẫu trực tiếp x_t từ x_0.

III.2. Ba dạng cấu trúc ma trận chuyển trạng thái

D3PM thiết lập ba cấu trúc ma trận chuyển Q_t tiêu biểu (với hệ số giữ tín hiệu tích lũy $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$):

1. Nhiễu đều (Uniform): Token giữ nguyên với xác suất $1 - \beta_t$ hoặc thay thế đều sang từ bất kỳ với xác suất β_t / K:

$$Q_t = (1 - \beta_t) I + \frac{\beta_t}{K} \mathbf{1}\mathbf{1}^T, \qquad \bar{Q}_t = \bar{\alpha}_t I + \frac{1 - \bar{\alpha}_t}{K} \mathbf{1}\mathbf{1}^T \tag{9}$$

Trong đó:

- $I \in \mathbb{R}^{K \times K}$ – ma trận đơn vị; $\mathbf{1} \in \mathbb{R}^K$ – vector cột toàn 1; $\beta_t \in (0,1)$ – tỷ lệ nhiễu bước t.

2. Trạng thái hấp thụ (Absorbing State / Mặt nạ [M]): Bổ sung token mặt nạ [M] (chỉ số K). Token sạch chuyển thành [M] với xác suất β_t và một khi đã là [M] thì giữ nguyên vĩnh viễn:

$$Q_t = \begin{bmatrix} (1-\beta_t) I_{K-1} & \beta_t \mathbf{1}_{K-1} \\ \mathbf{0}^T & 1 \end{bmatrix}, \qquad \bar{Q}_t = \begin{bmatrix} \bar{\alpha}_t I_{K-1} & (1-\bar{\alpha}_t) \mathbf{1}_{K-1} \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{10}$$

Trong đó:

- I_{K-1} – ma trận đơn vị $(K-1) \times (K-1)$; $\bar{\alpha}_t$ – xác suất giữ nguyên token sạch; $1 - \bar{\alpha}_t$ – xác suất bị che thành [M].

3. Nhiễu láng giềng (Discretized Gaussian): Phù hợp cho dữ liệu có thứ tự tự nhiên (như mức xám pixel), nhưng trên ngôn ngữ tổng quát, ma trận hấp thụ vượt trội hoàn toàn về

chất lượng hội tụ.

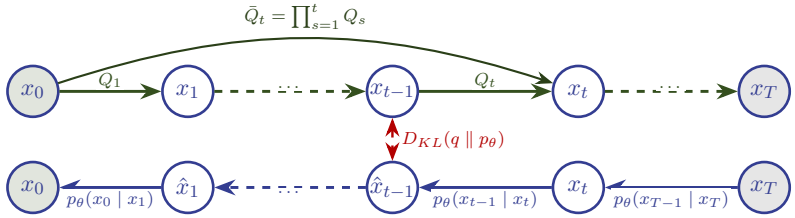
III.3. Phân phối hậu nghiệm ngược và Cận dưới biến phân (ELBO)

Áp dụng định lý Bayes $q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)}$, phân phối hậu nghiệm lùi khi biết x_0 có dạng đóng:

$$q(x_{t-1} \mid x_t, x_0) = \text{Cat}\left(x_{t-1};\ \frac{\mathbf{e}_{x_t} Q_t^T \odot \mathbf{e}_{x_0} \bar{Q}_{t-1}}{\mathbf{e}_{x_0} \bar{Q}_t \mathbf{e}_{x_t}^T}\right) \tag{11}$$

Trong đó:

- $\odot$ – tích Hadamard từng phần tử; $\mathbf{e}_{x_t} Q_t^T$ – cột thứ x_t của Q_t; $\mathbf{e}_{x_0} \bar{Q}_{t-1}$ – hàng thứ x_0 của $\bar{Q}_{t-1}$.
- $\mathbf{e}_{x_0} \bar{Q}_t \mathbf{e}_{x_t}^T = [\bar{Q}_t]_{x_0, x_t}$ – hằng số chuẩn hóa xác suất biên tại bước t.



Hình 2: Quy trình khuếch tán tiến $(Q_t, \bar{Q}_t)$ và khử nhiễu lùi (p_θ) tối ưu qua độ phân kỳ KL trong D3PM.

Mục tiêu huấn luyện D3PM tối thiểu hóa cận trên biến phân $\mathcal{L}_{vb}$ của negative log-likelihood:

$$\mathcal{L}_{vb} = \mathbb{E}_q \left[\underbrace{D_{KL}\left(q(x_T \mid x_0) \parallel p(x_T)\right)}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{KL}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t)\right)}_{L_{t-1}} \underbrace{- \log p_\theta(x_0 \mid x_1)}_{L_0} \right] \tag{12}$$

Trong đó:

- L_T – khớp tiên nghiệm ($L_T \approx 0$ khi $\bar{\alpha}_T \approx 0$); L_{t-1} – khớp phân phối hậu nghiệm lùi tại bước t; L_0 – tái tạo bước cuối.
- $D_{KL}(p \parallel q) = \sum_{k=1}^{K} \pi_k^{(1)} \log(\pi_k^{(1)} / \pi_k^{(2)})$ – độ phân kỳ Kullback-Leibler giữa hai phân phối Categorical.

III.4. Ví dụ tính tay D3PM hoàn chỉnh từng bước

Xét $K = 4$ trạng thái gồm $\{A, B, C\}$ (chỉ số $0, 1, 2$) và mặt nạ hấp thụ [M] (chỉ số 3) với $T = 2$ bước: $\beta_1 = 0{,}2000$, $\beta_2 = 0{,}3000$. Ma trận chuyển tích lũy $\bar{Q}_2 = Q_1 Q_2$ có hệ số giữ tín hiệu $\bar{\alpha}_2 = (1-0{,}2000)(1-0{,}3000) = 0{,}8000 \times 0{,}7000 = 0{,}5600$ và xác suất bị che $1-\bar{\alpha}_2 = 0{,}4400$. Với $x_0 = A$ ($\mathbf{e}_A = [1, 0, 0, 0]$), phân phối biên bước 2 là $q(x_2 \mid x_0 = A) = [0{,}5600,\ 0{,}0000,\ 0{,}0000,\ 0{,}4400]$.

Khi quan sát $x_2 =$ [M] và biết $x_0 = A$, tử số Bayes (11) cho $x_1 = A$ là $[Q_2]_{03}[Q_1]_{00} = 0{,}3000 \times 0{,}8000 = 0{,}2400$; cho $x_1 =$ [M] là $[Q_2]_{33}[Q_1]_{03} = 1{,}0000 \times 0{,}2000 = 0{,}2000$ (mẫu số

$0{,}2400 + 0{,}2000 = 0{,}4400$). Chuẩn hóa thu được $q(x_1 = A \mid x_2 = [\text{M}], x_0 = A) = \frac{0{,}2400}{0{,}4400} \approx 0{,}5455$ và $q(x_1 = [\text{M}] \mid x_2 = [\text{M}], x_0 = A) = \frac{0{,}2000}{0{,}4400} \approx 0{,}4545$ (Bảng 3).

Đại lượng xác suất	**Trạng thái A**	**Trạng thái B**	**Trạng thái C**	**Trạng thái [M]**
Phân phối gốc $\mathbf{e}_{x_0}$ ($x_0 = A$)	1,0000	0,0000	0,0000	0,0000
Phân phối biên bước 1: $q(x_1 \mid x_0 = A)$	0,8000	0,0000	0,0000	0,2000
Phân phối biên bước 2: $q(x_2 \mid x_0 = A)$	0,5600	0,0000	0,0000	0,4400
Tử số Bayes chưa chuẩn hóa ($x_2 = [\text{M}]$)	0,2400	0,0000	0,0000	0,2000
Hậu nghiệm $q(x_1 \mid x_2 = [\text{M}], x_0 = A)$	**0,5455**	**0,0000**	**0,0000**	**0,4545**

Bảng 3: Tổng kết số học phân phối biên tiến và hậu nghiệm lùi trong ví dụ tính tay D3PM hấp thụ ($K = 4$).

Vì sao trạng thái hấp thụ ([MASK]) áp đảo hoàn toàn trong mô hình ngôn ngữ?

Thực nghiệm D3PM [3] khẳng định ma trận hấp thụ (*Absorbing State*) vượt trội nhờ: (1) **Bảo toàn ngữ cảnh sạch:** token chưa bị che luôn chuẩn xác 100%, không gây trôi dạt ngữ nghĩa như nhiễu đều; (2) **Thu hẹp không gian dự đoán:** chỉ cần giải mã tại các vị trí [M]; (3) **Tương thích hoàn hảo với Transformer hai chiều** tương tự BERT [15].

Tóm lại, D3PM đã thiết lập khung toán học xích Markov mẫu mực cho khuếch tán rời rạc và làm sáng tỏ ưu thế của trạng thái hấp thụ. Phần IV. sẽ mở rộng khuếch tán rời rạc sang miền thời gian liên tục với phương pháp ước lượng tỷ số điểm số SEDD.

IV. Score-based Discrete Diffusion: Tiếp cận thời gian liên tục với SEDD

Trong khi D3PM phụ thuộc trục thời gian rời rạc cố định T bước, Lou et al. (2024) [4] đã đề xuất **SEDD** (*Score Entropy Discrete Diffusion*, đạt giải *Best Paper Award* tại ICML 2024), mở rộng khuếch tán rời rạc sang miền thời gian liên tục $t \in [0, 1]$ bằng cách kết hợp xích Markov thời gian liên tục (*Continuous-Time Markov Chains* – CTMC) với khái niệm *Concrete Score* và hàm mục tiêu *Score Entropy*.

IV.1. Xích Markov thời gian liên tục và ma trận tốc độ chuyển trạng thái

Trên từ vựng rời rạc $\mathcal{V}$ ($|\mathcal{V}| = K$), quá trình khuếch tán xuôi liên tục theo $t \in [0,1]$ được mô tả bởi một CTMC với ma trận tốc độ chuyển trạng thái (*transition rate matrix*) $R_t \in \mathbb{R}^{K\times K}$ thỏa mãn $R_t(y,x) \geq 0$ ($\forall x \neq y$) và $R_t(y,y) = -\sum_{x\neq y} R_t(y,x)$. Trong khoảng vi phân dt, xác suất nhảy trạng thái và phương trình vi phân chủ (*Master Equation*) có dạng:

$$q(x_{t+dt} = x \mid x_t = y) = R_t(y,x)\,dt + o(dt) \quad (x \neq y), \qquad \frac{d}{dt}\mathbf{p}_t = \mathbf{p}_t R_t, \quad q(x_t \mid x_0) = \mathbf{e}_{x_0}\exp(tR) \tag{13}$$

Trong đó:

- $\mathbf{p}_t = [p_t(1), \ldots, p_t(K)]$ – vector hàng phân phối xác suất tại thời điểm $t \in [0,1]$; dt – gia số thời gian vi phân.
- $R_t(y,x)$ – tốc độ chuyển tức thời từ y sang x; $\exp(tR) = \sum_{n=0}^{\infty} \frac{(tR)^n}{n!}$ – số mũ ma trận khi $R_t \equiv R$.

IV.2. Khái niệm Concrete Score và Hàm mục tiêu Score Entropy

Do phép vi phân $\nabla_x \log p_t(x)$ không tồn tại trên không gian rời rạc, Lou et al. (2024) [4] định nghĩa **Concrete Score** dưới dạng tỷ số xác suất biên, đóng vai trò điều khiển trực tiếp ma trận tốc độ của quá trình CTMC đảo ngược thời gian R_t^B:

$$s_t(y,x) = \frac{p_t(y)}{p_t(x)}, \qquad R_t^B(x,y) = R_t(y,x)\,\frac{p_t(y)}{p_t(x)} = R_t(y,x)\,s_t(y,x) \quad (x \neq y) \tag{14}$$

Trong đó:

- $s_t(y,x)$ – đại lượng Concrete Score giữa trạng thái nguồn x và trạng thái đích y tại thời điểm t.
- $p_t(y), p_t(x)$ – xác suất biên tại thời điểm t; $R_t^B(x,y)$ – tốc độ nhảy lùi từ x về y trong quá trình khử nhiễu.

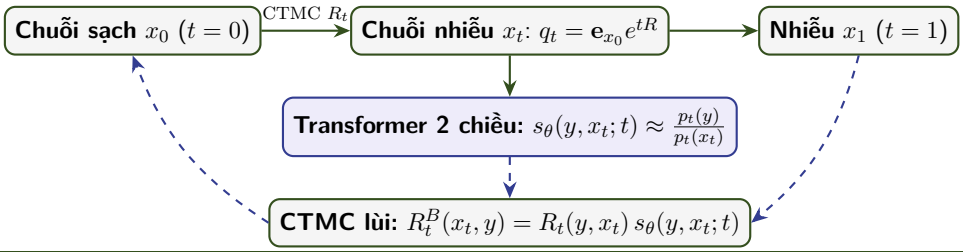
Để tránh bùng nổ tổ hợp trên chuỗi độ dài L, SEDD chỉ xét các bước nhảy lân cận Hamming bậc 1 $y = (x^{\setminus i}, k)$, đưa đầu ra mạng về kích thước $L \times K$. Nhằm huấn luyện mạng $s_\theta(y, x_t; t)$ ước lượng tỷ số $\frac{p_t(y)}{p_t(x_t)}$ luôn dương và khả quy, SEDD tối ưu hàm mất mát **Score Entropy**:

$$\mathcal{L}_{\text{SEDD}}(\theta) = \mathbb{E}_{\substack{t\sim\mathcal{U}(0,1),\, x_0\sim p_{\text{data}} \\ x_t\sim q_t(\cdot\mid x_0)}} \left[\sum_{y\neq x_t} R_t(x_t,y)\left(s_\theta(y,x_t;t) - \frac{q_t(y\mid x_0)}{q_t(x_t\mid x_0)}\log s_\theta(y,x_t;t)\right)\right] \tag{15}$$

Trong đó:

- $s_\theta(y,x_t;t) > 0$ – dự đoán của mạng Transformer hai chiều cho tỷ số Concrete Score.
- $s^* = \frac{q_t(y\mid x_0)}{q_t(x_t\mid x_0)}$ – tỷ số chuyển xuôi có điều kiện biết x_0. Hàm lồi $g(s) = s - s^*\log s$ đạt cực

tiểu duy nhất tại $s = s^*$, đảm bảo nghiệm tối ưu hội tụ đúng về $\mathbb{E}_{x_0|x_t}[s^*] = \frac{p_t(y)}{p_t(x_t)}$.



Hình 3: Quy trình khuếch tán thời gian liên tục SEDD dựa trên ước lượng Concrete Score s_θ và tốc độ lùi R_t^B.

IV.3. Ví dụ tính tay: Tính toán Concrete Score và Score Entropy Loss

Xét $K = 3$ trạng thái $\mathcal{V} = \{A, B, C\}$ với ma trận tốc độ đối xứng $R(x, y) = 1{,}0$ $(\forall x \neq y)$ và $R(x, x) = -2{,}0$. Với $t = 0{,}50$ và $x_0 = A$, nghiệm giải tích $P_{0,t} = \exp(tR)$ có $e^{-3t} = e^{-1{,}50} \approx 0{,}2231$, suy ra:

$$q_t(A \mid x_0 = A) = \frac{1 + 2e^{-1{,}50}}{3} \approx 0{,}4821, \quad q_t(B \mid x_0 = A) = q_t(C \mid x_0 = A) = \frac{1 - e^{-1{,}50}}{3} \approx 0{,}2590$$

Giả sử tại $t = 0{,}50$ quan sát thấy $x_t = B$. Tỷ số mục tiêu là $s_t^*(A, B) = \frac{0{,}4821}{0{,}2590} \approx 1{,}8617$ và $s_t^*(C, B) = \frac{0{,}2590}{0{,}2590} = 1{,}0000$. Nếu mạng dự đoán $s_\theta(A, B) = 1{,}8000$ và $s_\theta(C, B) = 1{,}1000$, các số hạng tổn thất $\ell(y) = R_t(B, y)[s_\theta - s^* \ln s_\theta]$ là $\ell(A) = 1{,}8000 - 1{,}8617 \ln(1{,}8000) \approx 0{,}7057$ và $\ell(C) = 1{,}1000 - 1{,}0000 \ln(1{,}1000) \approx 1{,}0047$, tổng tổn thất $\mathcal{L}_{\text{SEDD}} = 1{,}7104$ (Bảng 4).

Trạng thái y	$q_t(y \mid x_0)$	$s_t^*(y, B)$	$s_\theta(y, B)$	$\ln s_\theta$	$s^* \ln s_\theta$	$\ell(y)$
A (Gốc)	0,4821	1,8617	1,8000	0,5878	1,0943	0,7057
C (Nhiễu)	0,2590	1,0000	1,1000	0,0953	0,0953	1,0047
Tổng tổn thất Score Entropy tại $x_t = B$**:**						**1,7104**

Bảng 4: Các bước tính toán trung gian của hàm mất mát SEDD tại $x_t = B$ với $t = 0{,}50$.

Tóm lại, SEDD đã thiết lập cầu nối lý thuyết giữa khuếch tán thời gian liên tục và không gian ngôn ngữ rời rạc qua tỷ số Concrete Score $s_t(y, x) = \frac{p_t(y)}{p_t(x)}$. Khi chuyên biệt hóa sang trạng thái hấp thụ duy nhất `[M]`, cận dưới biến phân rút gọn thành hàm mất mát cross-entropy quen thuộc của **MDLM/MD4**, được trình bày trong Phần V.

V. Masked Discrete Diffusion: Cơ chế che dần rồi đoán lại và tối ưu hóa MDLM/MD4

Khi các token bị thay thế ngẫu nhiên thành những từ khác trong từ vựng (như nhiễu đều), chuỗi trung gian bị trôi dạt ngữ nghĩa nghiêm trọng. Để khắc phục, nhánh tiếp cận **Masked**

Discrete Diffusion – tiêu biểu bởi MDLM của Sahoo et al. (NeurIPS 2024) [5], MD4 của Shi et al. (NeurIPS 2024) [6] và RADD của Ou et al. (ICLR 2025) [16] – giới hạn quá trình nhiễu chỉ biến đổi token thành một trạng thái hấp thụ duy nhất `[M]`, biến bài toán khử nhiễu thành điền khuyết có cấu trúc (*infilling*) và rút gọn cận dưới ELBO thành hàm mất mát cross-entropy có trọng số.

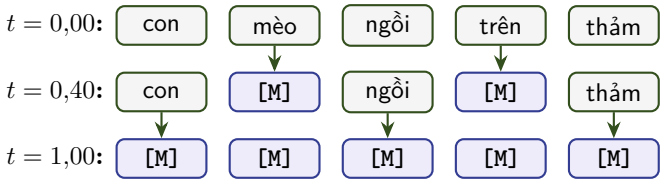
V.1. Quá trình khuếch tán xuôi hấp thụ và Tỷ lệ rủi ro (Hazard Rate)

Xét chuỗi sạch $x_0 = (x_0^1, \ldots, x_0^L) \in \mathcal{V}^L$ mở rộng thêm token mặt nạ `[M]` với lịch trình giữ tín hiệu $\alpha_t \in [0,1]$ ($\alpha_0 = 1, \alpha_1 = 0$). Phân phối biên xuôi tại bước t và xác suất mở khóa mặt nạ trong bước lùi vi phân dt là:

$$q(x_t^i \mid x_0^i) = \alpha_t \, \mathbf{e}_{x_0^i} + (1 - \alpha_t) \, \mathbf{e}_{\texttt{[M]}}, \qquad q(x_{t-dt}^i = x_0^i \mid x_t^i = \texttt{[M]}, x_0^i) = \frac{-\alpha_t'}{1 - \alpha_t} \, dt = \lambda(t) \, dt \tag{16}$$

Trong đó:

- α_t – xác suất giữ nguyên token sạch x_0^i; $1 - \alpha_t$ – xác suất bị che bởi `[M]`.
- $\lambda(t) = \frac{-\alpha_t'}{1-\alpha_t}$ – hàm tỷ lệ rủi ro (*hazard rate*) của sự kiện mở mặt nạ. Với lịch trình tuyến tính $\alpha_t = 1 - t$ ($\alpha_t' = -1$), ta có $\lambda(t) = \frac{1}{t}$.



Hình 4: Quá trình khuếch tán xuôi hấp thụ: Token chuyển dần thành `[M]` theo $t \in [0,1]$ và bị khóa ở trạng thái hấp thụ.

V.2. Đột phá lý thuyết: Đơn giản hóa ELBO thành Weighted Masked Cross-Entropy

Sahoo et al. (MDLM, 2024) [5] và Shi et al. (MD4, 2024) [6] chứng minh rằng với quá trình hấp thụ: tại các vị trí sạch ($x_t^i \neq \texttt{[M]}$), độ phân kỳ KL đồng nhất bằng 0 ($D_{\mathrm{KL}} \equiv 0$); tại các vị trí bị che ($x_t^i = \texttt{[M]}$), số hạng KL duy nhất còn lại là $\lambda(t)dt[-\log p_\theta(x_0^i \mid x_t)]$. Tích phân trên $t \in [0,1]$ với $\alpha_t = 1 - t$ đưa cận dưới ELBO về dạng:

$$\mathcal{L}_{\mathrm{MDLM}}(\theta) = -\mathbb{E}_{\substack{t \sim \mathcal{U}(0,1),\, x_0 \sim p_{\mathrm{data}} \\ x_t \sim q(x_t \mid x_0)}} \left[\frac{1}{t} \sum_{i:\, x_t^i = \texttt{[M]}} \log p_\theta(x_0^i \mid x_t) \right] \tag{17}$$

Trong đó:

- $t \sim \mathcal{U}(0,1)$ – mức nhiễu; $\frac{1}{t} = \lambda(t)$ – trọng số thời gian cân bằng phương sai gradient (do kỳ vọng số vị trí bị che là $L \cdot t$).

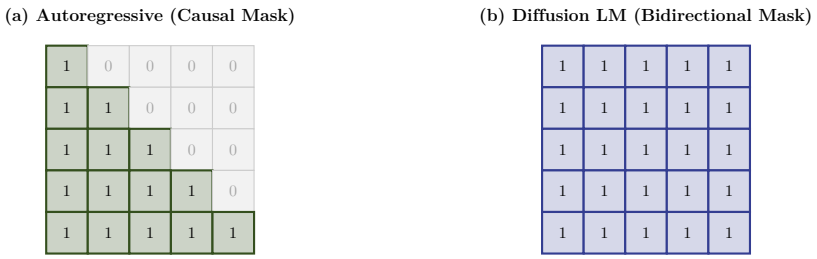
- $p_\theta(x_0^i \mid x_t)$ – xác suất dự đoán token gốc x_0^i tại vị trí bị che i dựa trên ngữ cảnh hai chiều x_t.

Tiêu chí	BERT / MLM (Devlin et al., 2019)	Masked Diffusion (MDLM / MD4)
Tỷ lệ che mặt nạ	Cố định 15%	Liên tục ngẫu nhiên $t \sim \mathcal{U}(0,1)$ (0% $\rightarrow$ 100%)
Trọng số mất mát	Trọng số đều 1,0 (Encoder trích đặc trưng)	Trọng số giải tích $\frac{1}{t}$ từ ELBO (Mô hình sinh)
Quỹ đạo giải mã	Không có cơ chế khử nhiễu toán học	Giải mã CTMC từ $x_1 = \texttt{[M]}^L$ về x_0 qua T bước

Bảng 5: Đối chiếu bản chất giữa Masked Language Modeling (BERT) và Masked Discrete Diffusion (MDLM/MD4).

V.3. Cơ chế Attention: Causal Attention vs Full Bidirectional Attention

Khác với mô hình AR bị giới hạn bởi mặt nạ tam giác nhân quả (*Causal Mask*), Masked Diffusion áp dụng **chú ý hai chiều toàn cục** (*Full Bidirectional Attention*) trên toàn bộ chuỗi x_t (Hình 5), mang lại hai năng lực tự nhiên: (1) **Điền khuyết hai đầu (Infilling)** không cần sắp xếp lại dữ liệu; và (2) **Khắc phục triệt để Lời nguyền đảo ngược (Reversal Curse)**.



Hình 5: Ma trận Attention Mask 5×5: (a) AR chỉ chú ý quá khứ tam giác dưới; (b) DLM chú ý hai chiều toàn cục.

V.4. Ví dụ tính tay: Tính toán hàm mất mát MDLM/MD4 trên câu 5 token

Xét chuỗi $L = 5$ token $x_0 =$ (“con”, “mèo”, “ngồi”, “trên”, “thảm”) tại $t = 0{,}4000$ ($\alpha_t = 0{,}6000$, trọng số $w(t) = \frac{1}{0{,}4000} = 2{,}5000$). Giả sử vị trí $i = 2$ và $i = 4$ bị che:

$$x_t = (\text{“con”}, \texttt{[M]}, \text{“ngồi”}, \texttt{[M]}, \text{“thảm”}), \tag{18}$$

với dự đoán $p_\theta(x_0^2 = \text{“mèo”} \mid x_t) = 0{,}7000$ và $p_\theta(x_0^4 = \text{“trên”} \mid x_t) = 0{,}5000$. Tổn thất tại hai vị trí bị che là $\text{CE}_2 = -\ln(0{,}7000) \approx 0{,}3567$ và $\text{CE}_4 = -\ln(0{,}5000) \approx 0{,}6931$ (tổng 1,0498). Nhân trọng số $w(t) = 2{,}5000$ thu được $\mathcal{L}_{\text{MDLM}} = 2{,}5000 \times 1{,}0498 = 2{,}6246$, tức trung bình mỗi token $\mathcal{L}_{\text{per-token}} = \frac{2{,}6246}{5} \approx 0{,}5249$ (Bảng 6).

Vị trí i	**Token** x_0^i	**Trạng thái** x_t^i	**Mặt nạ?**	$p_\theta(x_0^i \mid x_t)$	$\text{CE}_i = -\ln p_\theta$	**Đóng góp**
1, 3, 5	“con”, “ngồi”, “thảm”	Giữ nguyên	Không	–	–	0,0000
2	“mèo”	[M]	Có	0,7000	0,3567	0,3567
4	“trên”	[M]	Có	0,5000	0,6931	0,6931
Tổng CE vị trí bị che (1,0498) × Trọng số $w(t) = 2{,}5000$**:**						**2,6246**

Bảng 6: Tính toán hàm mất mát MDLM trên câu 5 token tại mức nhiễu $t = 0{,}40$ (trung bình **0,5249**/token).

Tóm lại, cơ chế Masked Discrete Diffusion (MDLM, MD4, RADD) biến cận dưới ELBO thành bài toán cross-entropy trọng số $\frac{1}{t}$ tương thích hoàn hảo với Transformer hai chiều. Phần VI. sẽ khảo sát cách mở rộng nguyên lý này lên quy mô hàng tỷ tham số với LLaDA, Dream cùng các chiến lược lấy mẫu tiên tiến.

VI. Mở rộng quy mô và thuật toán lấy mẫu: LLaDA, Dream và Remasking

Giai đoạn 2025–2026 đánh dấu bước chuyển mình của các mô hình ngôn ngữ khuếch tán lên quy mô hàng tỷ tham số (*Frontier DLMs*) như LLaDA-8B [2] và Dream-7B [7]. Kết hợp giữa kiến trúc Transformer hai chiều cùng các thuật toán lấy mẫu như *Classifier-Free Guidance*, *Low-Confidence Remasking* và *Block Diffusion*, DLM đã đạt chất lượng cạnh tranh trực tiếp với các LLM tự hồi quy mạnh nhất.

VI.1. Các mô hình nền tảng khuếch tán quy mô lớn (Frontier DLMs)

1. LLaDA 8B – Huấn luyện từ đầu (Pre-training from scratch): LLaDA (*Large Language Diffusion with mAsking*, Nie et al., 2025 [2]) là mô hình khuếch tán rời rạc 8B đầu tiên huấn luyện từ đầu trên 2,3T token với cơ chế tự chú ý hai chiều toàn cục (RoPE, RMSNorm, SwiGLU). Ở giai đoạn SFT, LLaDA giữ nguyên 100% câu hỏi và chỉ che ngẫu nhiên phần phản hồi theo $t \sim \mathcal{U}(0, 1)$, qua đó khắc phục triệt để “Lời nguyền Đảo ngược” (*Reversal Curse*).

2. Dream 7B – Chuyển giao trọng số từ AR (AR Weight Transfer): Dream-7B (Ye et al., 2025 [7], HKU/Huawei) khởi tạo trực tiếp từ Qwen2.5-7B, mở rộng mặt nạ tam giác thành

chú ý hai chiều và áp dụng *context-adaptive token-level noise rescheduling*, đạt chất lượng tương đương LLaMA-3 8B chỉ với $< 5\%$ chi phí huấn luyện của LLaDA.

3. DiffuCoder và Mercury Coder: DiffuCoder-7B (Gong et al., 2025 [8]) huấn luyện trên 130B token mã nguồn với thuật toán coupled-GRPO; Mercury Coder [11] đạt thông lượng $> 1\,000$ token/giây nhờ giải mã thô-đến-tinh đa bước (Bảng 7).

Mô hình	Tổ chức	Năm	Quy mô	Đặc điểm kiến trúc & Đột phá	Nguồn
LLaDA	ML-GSAI	2025	8B	Huấn luyện từ đầu (2,3T tok), SFT phản hồi, giải Reversal Curse	[2]
Dream	HKU/Huawei	2025	7B	Chuyển trọng số từ Qwen2.5-7B, tái lập lịch nhiễu ngữ cảnh	[7]
DiffuCoder	Apple	2025	7B	Coupled-GRPO, mặt nạ bù trừ sửa mã lệnh cục bộ	[8]
Mercury	Inception	2025	Thương mại	Giải mã thô-đến-tinh đa bước, thông lượng $> 1\,000$ tok/s	[11]

Bảng 7: Tổng hợp các mô hình Diffusion Language Models quy mô lớn tiêu biểu (2025–2026).

Lưu ý

Lưu ý thực nghiệm benchmark: Hiệu năng benchmark (GSM8K, HumanEval, MMLU) của DLM phụ thuộc chặt chẽ vào số bước lấy mẫu S và thiết lập đánh giá; người đọc cần lưu ý đối chiếu công bằng khi điều kiện suy luận khác nhau.

VI.2. Thuật toán lấy mẫu và Classifier-Free Guidance trong miền rời rạc

Kỹ thuật Hướng dẫn Không bộ Phân loại (*Classifier-Free Guidance* – CFG) [2, 7] kết hợp vector logit có điều kiện ngữ cảnh c và không điều kiện $\emptyset$, sau đó lấy mẫu qua hàm Softmax nhiệt độ τ:

$$\tilde{\ell}_\theta(x_t, c) = \ell_\theta(x_t, \emptyset) + s \cdot \Big(\ell_\theta(x_t, c) - \ell_\theta(x_t, \emptyset)\Big), \qquad \tilde{p}_\theta(x_0^i = k \mid x_t, c) = \frac{\exp\left(\tilde{\ell}_\theta^i(k, x_t, c)/\tau\right)}{\sum_{j=1}^{K} \exp\left(\tilde{\ell}_\theta^i(j, x_t, c)/\tau\right)} \tag{19}$$

Trong đó:

- $\ell_\theta(x_t, c), \ell_\theta(x_t, \emptyset)$ – logit có điều kiện c và không điều kiện; $s \geq 1$ – hệ số hướng dẫn (*guidance scale*).
- $\tilde{p}_\theta(x_0^i = k \mid x_t, c)$ – xác suất gán token k tại vị trí i; $\tau > 0$ – nhiệt độ điều khiển tính đa dạng.

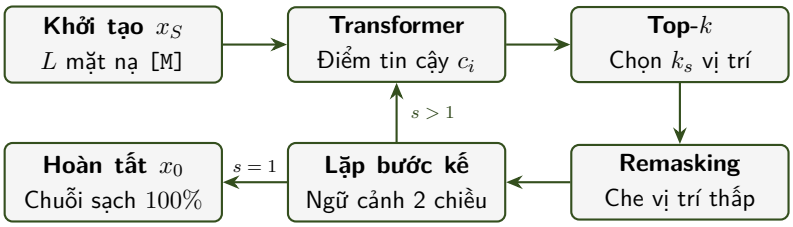
VI.3. Chiến lược Che lại theo Độ tin cậy (Low-Confidence Remasking)

Để ngăn ngừa tích tụ sai số (*error cascading*) ở các bước đầu, LLaDA [2] áp dụng **Low-Confidence Remasking**: tại bước suy luận $s \in \{S, \dots, 1\}$, mô hình chỉ giữ lại k_s vị trí có điểm tin cậy c_i cao nhất và gán lại `[M]` (*remask*) cho các vị trí còn lại:

$$c_i = \max_{k \in \mathcal{V}} p_\theta(x_0^i = k \mid x_s), \qquad k_s = \left\lfloor L \cdot \frac{S - s + 1}{S} \right\rfloor \tag{20}$$

Trong đó:

- c_i – điểm tin cậy tại vị trí i; k_s – ngân sách token sạch tích lũy trên chuỗi độ dài L tại bước s (trên tổng S bước).



Hình 6: Quy trình lấy mẫu suy luận lặp với thuật toán Low-Confidence Remasking.

Ví dụ tính tay Remasking ($L = 4$ token, $S = 2$ bước) cho chuỗi mục tiêu ("Hà", "Nội", "mùa", "thu"): ở $s = 2$, điểm tin cậy là (0,8500, 0,9000, 0,4500, 0,3000) với ngân sách $k_2 = 2$, mô hình giữ vị trí 1, 2 và remask vị trí 3, 4 thành ("Hà", "Nội", `[M]`, `[M]`). Ở $s = 1$, nhờ ngữ cảnh "Hà Nội", điểm tin cậy vị trí 3, 4 tăng lên 0,9200 và 0,8800 ($k_1 = 4$), hoàn tất chuỗi sạch (Bảng 8).

Bước (s)	Vị trí 1	Vị trí 2	Vị trí 3	Vị trí 4	Quyết định & Chuỗi trung gian
Khởi đầu	`[M]`	`[M]`	`[M]`	`[M]`	Khởi tạo x_2 toàn mặt nạ
$s = 2$	0,8500	0,9000	0,4500	0,3000	Giữ 1, 2; remask 3, 4 → ("Hà", "Nội", `[M]`, `[M]`)
$s = 1$	0,9800	0,9900	0,9200	0,8800	Mở toàn bộ → ("Hà", "Nội", "mùa", "thu")

Bảng 8: Diễn biến điểm tin cậy và quyết định unmask/remask qua 2 bước suy luận.

VI.4. Khuếch tán theo Khối (Block Diffusion - BD3-LM)

Để khắc phục hạn chế chiều dài cố định và tái sử dụng KV Cache, Arriola et al. (ICLR 2025) [9] đề xuất **Block Diffusion** (BD3-LM): chia chuỗi L thành M khối kích thước B ($L = M \cdot B$), kết hợp **nhân quả giữa các khối** (*Inter-block Causal Attention*) và **khuếch tán hai chiều nội khối** (*Intra-block Bidirectional Diffusion*) trong S_b bước (Hình 7). Khi khối m hoàn tất, Key và Value của nó được đóng băng vĩnh viễn cho các khối sau.



Hình 7: Ma trận chú ý tam giác theo khối (Block Lower-Triangular Mask) trong Block Diffusion.

Tóm lại, LLaDA, Dream cùng các thuật toán CFG, Low-Confidence Remasking và Block Diffusion đã đưa DLM lên vị thế mô hình nền tảng. Phần VII. sẽ phân tích bài toán KV Cache, giải pháp tăng tốc Fast-dLLM và chứng minh toán học về sự thất bại của giả định độc lập khi giải mã song song.

VII. Tăng tốc suy luận và kiến trúc hệ thống: Thách thức KV-Cache và Fast-dLLM

Trong các hệ thống phục vụ mô hình tự hồi quy (AR), kỹ thuật bộ nhớ đệm khóa–giá trị (*KV Cache*) giữ vai trò trụ cột nhờ mặt nạ nhân quả tam giác. Ngược lại, trong Diffusion Language Models, cơ chế chú ý hai chiều toàn cục tạo nên nghịch lý kiến trúc: mỗi token được gỡ mặt nạ ở bước mới đều làm thay đổi trạng thái ẩn của toàn bộ các vị trí khác, khiến KV Cache cổ điển bị vô hiệu hóa. Phần này phân tích nghịch lý KV Cache, giải pháp tăng tốc từ Fast-dLLM [10], và chứng minh toán học về sự thất bại của giả định độc lập có điều kiện cùng cơ chế phục hồi của DLM.

VII.1. Nghịch lý KV Cache trong cơ chế chú ý hai chiều

Trong mô hình AR, vector chú ý tại bước $i+1$ chỉ phụ thuộc vào $K_{\leq i+1}, V_{\leq i+1}$ bất biến của quá khứ (chi phí $\mathcal{O}(i)$ mỗi bước). Trái lại, trong DLM với chú ý hai chiều toàn cục, trạng thái ẩn tại vị trí j, tầng l ở bước khử nhiễu t phụ thuộc vào toàn bộ L vị trí:

$$\text{Attention}_{\text{AR}}(Q_{i+1}, K_{\leq i+1}, V_{\leq i+1}) = \text{softmax}\left(\frac{Q_{i+1}K_{\leq i+1}^T}{\sqrt{d_k}}\right)V_{\leq i+1}, \qquad \mathbf{h}_{j,l}^{(t)} = \sum_{k=1}^{L} \alpha_{j,k}^{(t)} W_V \mathbf{h}_{k,l-1}^{(t)} \tag{21}$$

Trong đó:

- $Q_{i+1} \in \mathbb{R}^{1\times d_k}$ và $K_{\leq i+1}, V_{\leq i+1}$ – Query mới và bộ nhớ đệm KV tích lũy trong AR (d_k là chiều mỗi đầu chú ý).
- $\mathbf{h}_{j,l}^{(t)}$ – trạng thái ẩn vị trí j, tầng l ở bước khuếch tán t; $\alpha_{j,k}^{(t)}$ – trọng số chú ý hai chiều; W_V – ma trận chiếu Value.

Khi một vị trí chuyển từ `[M]` sang token sạch ở bước $t-1$, toàn bộ $\mathbf{h}_k^{(t-1)}$ biến thiên dây chuyền, buộc DLM thuần túy phải tính lại toàn bộ đồ thị chú ý với chi phí $S \cdot \mathcal{O}(L^2)$.

VII.2. Các giải pháp xấp xỉ kiến trúc hệ thống và Fast-dLLM

Kiến trúc **Fast-dLLM** của Wu et al. (ICLR 2026) [10], kết hợp cùng Block Diffusion (BD3-LM) [9], giải quyết nút thắt này qua bốn cơ chế:

1. **Lưu đệm tiền tố tĩnh (Static Prefix-Caching):** Tính trước và đóng băng KV Cache của vùng Prompt (không chứa `[M]`).
2. **Lưu đệm theo bước (Step-Caching / dLLM-Cache):** Tái sử dụng KV Cache cho các token có độ trôi biểu diễn nhỏ ($\|\mathbf{h}_t - \mathbf{h}_{t-1}\| < \epsilon$).
3. **Lưu đệm theo khối (Block-wise KV Cache):** Đóng băng 100% KV Cache của các khối đã hội tụ, chỉ tính động trong khối hiện hành.
4. **Giải mã song song nhận biết độ tự tin (Confidence-Aware Parallel Decoding & Remasking):** Giữ nguyên các vị trí đạt ngưỡng độ tự tin cao ($\max_k p(k) > 0{,}95$) và áp dụng remasking cho các vị trí có độ tự tin thấp.

Ví dụ định lượng tiết kiệm FLOPs chú ý ($L = 512, B = 32, M = 16$ khối): So sánh DLM toàn cục không lưu đệm ($S_{\text{full}} = 32$ bước) và Block Diffusion có lưu đệm ($S_b = 4$ bước/khối):

$$\begin{aligned} \text{Ops}_{\text{Full}} &= S_{\text{full}} L^2 = 32 \times 512^2 = 8\,388\,608, \\ \text{Ops}_{\text{BD}} &= \sum_{m=0}^{M-1} S_b B^2 (m+1) = S_b B^2 \frac{M(M+1)}{2} = 557\,056 \end{aligned} \tag{22}$$

Trong đó:

- $\text{Ops}_{\text{Full}}, \text{Ops}_{\text{BD}}$ – tổng số phép tính tương tác chú ý của DLM toàn cục và Block Diffusion có KV Cache.
- $S_{\text{full}} = 32, S_b = 4$ – số bước suy luận; $L = 512, B = 32, M = 16$ – chiều dài chuỗi, kích thước khối và số khối. Tỉ số tăng tốc đạt Speedup $= \frac{8\,388\,608}{557\,056} \approx 15{,}06\times$ (cắt giảm 93,4%, Bảng 9).

Kiến trúc suy luận	Số bước	Cơ chế KV Cache	Tổng phép tính chú ý	Tỉ lệ cắt giảm
DLM toàn cục (Naïve)	$S = 32$	Không lưu đệm	8 388 608	0,0% (Gốc)
Block Diffusion (BD3-LM)	$S_b = 4$	Causal block cache	557 056	93,4% (15,06×)

Bảng 9: So sánh chi phí tương tác chú ý giữa DLM toàn cục và Block Diffusion có KV Cache ($L = 512, B = 32$).

VII.3. Bằng chứng toán học: Thất bại của giả định độc lập có điều kiện

Giải mã song song một bước ngây thơ giả định các vị trí độc lập có điều kiện $q_{\text{indep}}(x_1, \ldots, x_L \mid c) = \prod_{i=1}^{L} p(x_i \mid c)$, dẫn đến thất bại nghiêm trọng trên phân phối đa mode (*Conditional Independence Failure*).

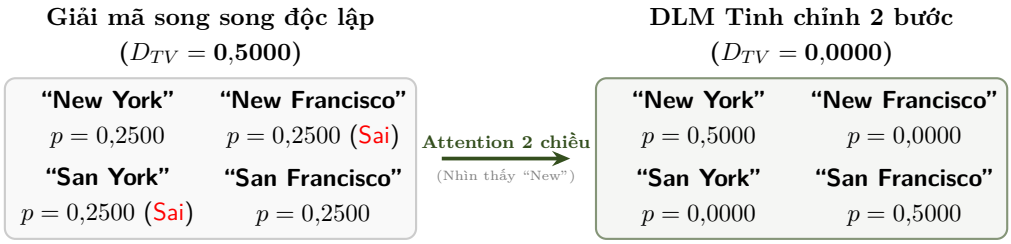
Phản ví dụ "New York" vs "San Francisco": Giả sử phân phối mục tiêu gồm hai cụm từ có xác suất đồng đều $P_{\text{true}}(\text{"New"}, \text{"York"}) = 0{,}5000$ và $P_{\text{true}}(\text{"San"}, \text{"Francisco"}) = 0{,}5000$, đồng thời bằng 0,0000 cho hai tổ hợp chéo. Phân phối biên tại mỗi vị trí đều bằng 0,5000. Giải mã độc lập $q_{\text{indep}} = P(\text{pos}_1)P(\text{pos}_2)$ san đều xác suất 0,2500 cho cả 4 tổ hợp, tạo ra 50,00% **tổ hợp lai tạp sai lệch (chimeric errors: "New Francisco", "San York")** với khoảng cách biến thiên toàn phần (D_{TV}):

$$D_{TV}(P_{\text{true}}, q_{\text{indep}}) = \frac{1}{2} \sum_{x \in \mathcal{X}} \left| P_{\text{true}}(x) - q_{\text{indep}}(x) \right| = \frac{1}{2}(4 \times 0{,}2500) = \mathbf{0{,}5000} \tag{23}$$

Trong đó:

- $D_{TV}(P, Q) \in [0, 1]$ – khoảng cách biến thiên toàn phần; $\mathcal{X}$ – tập 4 tổ hợp cặp từ khả dĩ.

Khôi phục hoàn hảo bằng DLM 2 bước: Ở bước $s = 2$ từ ([M], [M]), mô hình mở khóa vị trí 1 (ví dụ "New" với xác suất 0,5000). Ở bước $s = 1$, nhờ chú ý hai chiều nhìn thấy "New", phân phối có điều kiện tại vị trí 2 hội tụ tuyệt đối $p_\theta(\text{York} \mid \text{New}) = 1{,}0000$, đưa khoảng cách về $D_{TV}(P_{\text{true}}, P_{\text{DLM}}) = \mathbf{0{,}0000}$ (Hình 8 và Bảng 10).



Hình 8: So sánh phân phối đồng thời: Giải mã độc lập ($D_{TV} = 0{,}5000$) so với DLM tinh chỉnh 2 bước ($D_{TV} = 0{,}0000$).

Tổ hợp token	**Thực P_{true}**	**Độc lập q_{indep}**	**DLM 2 bước P_{DLM}**	**Đánh giá ngữ nghĩa**
("New", "York")	0,5000	0,2500	0,5000	Địa danh chuẩn xác
("New", "Francisco")	0,0000	0,2500	0,0000	Lỗi lai tạp (Chimeric)
("San", "York")	0,0000	0,2500	0,0000	Lỗi lai tạp (Chimeric)
("San", "Francisco")	0,5000	0,2500	0,5000	Địa danh chuẩn xác
Khoảng cách D_{TV}	0,0000 (Chuẩn)	**0,5000**	**0,0000**	Khôi phục hoàn hảo

Bảng 10: Phân phối xác suất và độ đo Total Variation Distance giữa phân phối thực, giải mã độc lập và DLM.

VII.4. Đánh đổi không gian–thời gian và hệ sinh thái phục vụ công nghiệp

Khi triển khai thực tế trên GPU, DLM chuyển dịch bài toán suy luận từ nghẽn băng thông bộ nhớ (*memory-bound* trong AR) sang tối ưu hóa tính toán (*compute-bound*) nhờ xử lý song song L vị trí trên Tensor Cores, với sự đánh đổi linh hoạt giữa số bước lấy mẫu S và chất lượng Perplexity.

Tóm lại, việc kết hợp tinh chỉnh hai chiều để triệt tiêu lỗi độc lập có điều kiện ($D_{TV} = 0{,}0000$) cùng các kỹ thuật lưu đệm xấp xỉ Fast-dLLM và Block Diffusion (cắt giảm 93,4% FLOPs chú ý) đã mở đường cho hạ tầng tạo sinh văn bản song song tốc độ cao. Phần VIII. sẽ hiện thực hóa toàn bộ nguyên lý này qua mô hình thực nghiệm PyTorch từ đầu.

VIII. Thực hành mô phỏng: Xây dựng và huấn luyện Mini-DLM với PyTorch

Nhằm hiện thực hóa các nguyên lý toán học của Masked Diffusion, phần này xây dựng mô hình ngôn ngữ khuếch tán mini (**Mini-DLM**) từ đầu (*from scratch*) bằng PyTorch thuần túy, đi kèm sổ tay tương tác `case_study/mini_dlm_case_study.ipynb` (đã lưu đầy đủ log đầu ra) và kịch bản độc lập `case_study/mini_dlm_case_study.py`.

VIII.1. Thiết kế kiến trúc mô hình và ngữ liệu sư phạm

Tập ngữ liệu sư phạm tiếng Việt gồm các câu độ dài cố định $L = 8$ token trên từ vựng $K = 44$ từ (gồm `[PAD]`, `[M]`, `[UNK]`) với cấu trúc ngữ pháp hai chiều chặt chẽ (ví dụ `thư_viện` đi kèm `đọc sách`, `công_sở` đi kèm `làm việc`):

$$\text{Mẫu câu} = [\text{Chủ ngữ}] + \text{“đi”} + [\text{Phương tiện 1, 2}] + \text{“đến”} + [\text{Địa điểm}] + [\text{Hành động}] + [\text{Đối tượng}] \tag{24}$$

Kế thừa thiết kế của MDLM [5] và LLaDA [2], mạng nhận chuỗi $x_t \in \mathbb{R}^{B\times L}$ và mức thời gian $t \in [0, 1]$, kết hợp nhúng token, vị trí và thời gian Sinusoidal ($\mathbf{h}_0^i = \mathbf{E}_{\text{tok}}(x_t^i) + \mathbf{E}_{\text{pos}}(i) + \mathbf{e}_{\text{time}}(t)$) qua khối Transformer Encoder Pre-LayerNorm với **chú ý hai chiều toàn phần** (*full bidirectional attention*).

Đặc tả kiến trúc MiniMaskedDiffusionLM trong PyTorch

```
import math, torch
import torch.nn as nn
```

```python
import torch.nn.functional as F

class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.d_model = d_model
        self.mlp = nn.Sequential(nn.Linear(d_model, d_model * 2), nn.SiLU(), nn.Linear(d_model * 2, d_model))
    def forward(self, t):
        half = self.d_model // 2
        emb = torch.exp(torch.arange(half, dtype=torch.float32, device=t.device) * -(math.log(10000) / max(half - 1, 1)))
        emb = t[:, None] * emb[None, :]
        return self.mlp(torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1))

class MiniMaskedDiffusionLM(nn.Module):
    def __init__(self, vocab_size, d_model=64, nhead=4, num_layers=2, dim_ff=128, max_len=16):
        super().__init__()
        self.token_emb, self.pos_emb = nn.Embedding(vocab_size, d_model), nn.Embedding(max_len, d_model)
        self.time_emb = SinusoidalTimeEmbedding(d_model)
        layer = nn.TransformerEncoderLayer(d_model, nhead, dim_ff, dropout=0.1, batch_first=True, norm_first=True)
        self.transformer = nn.TransformerEncoder(layer, num_layers=num_layers)
        self.head = nn.Linear(d_model, vocab_size)
    def forward(self, x, t):
        b, l = x.shape
        pos = torch.arange(l, device=x.device).unsqueeze(0).expand(b, l)
        return self.head(self.transformer(self.token_emb(x) + self.pos_emb(pos) + self.time_emb(t).unsqueeze(1)))
```

VIII.2. Quy trình huấn luyện và tối ưu hóa hàm mất mát

Mỗi bước huấn luyện lấy mẫu $t \sim \mathcal{U}(0,1)$, che độc lập từng vị trí thành `[M]` với xác suất $p_{\text{mask}} = t$ ($\alpha_t = 1 - t$) và tối ưu hàm mất mát Cross-Entropy chỉ trên các vị trí bị che ($m_i = \mathbb{I}[x_t^i = \texttt{[M]}]$):

$$\mathcal{L}_{\text{train}}(\theta) = -\frac{1}{\sum_{i=1}^{L} m_i} \sum_{i=1}^{L} m_i \log p_\theta(x_0^i \mid x_t, t) \tag{25}$$

Vòng lặp huấn luyện Mini-DLM với Forward Masking ngẫu nhiên

```
def train_epoch(model, dataloader, optimizer, scaler, device, use_amp):
    model.train()
    total_loss = 0.0
    for batch_x0 in dataloader:
        batch_x0 = batch_x0.to(device)
        b, l = batch_x0.shape
        t = torch.rand(b, device=device)
        is_masked = torch.rand((b, l), device=device) < t.unsqueeze(1)
        is_masked[:, torch.randint(0, l, (b,), device=device)] = True
        batch_xt = batch_x0.clone()
        batch_xt[is_masked] = MASK_IDX
        optimizer.zero_grad()
        with torch.amp.autocast('cuda', enabled=use_amp):
            logits = model(batch_xt, t)
            loss = F.cross_entropy(logits[is_masked], batch_x0[is_masked
    ])
        scaler.scale(loss).backward() if use_amp else loss.backward()
        scaler.step(optimizer) if use_amp else optimizer.step()
        if use_amp: scaler.update()
        total_loss += loss.item()
    return total_loss / len(dataloader)
```

Với cấu hình chuẩn hóa (`QUICK_RUN = True`, 123 692 tham số, `seed = 42`), mô hình trên PyTorch hội tụ sau khoảng 13 giây: hàm mất mát giảm từ 2,6554 (epoch 1) xuống 0,8096 (epoch 10) và đạt 0,6983 tại epoch 40.

VIII.3. Hành trình khử mặt nạ từng bước và Điền khuyết hai đầu

Quá trình suy luận áp dụng **Low-Confidence Remasking** [2]: xuất phát từ $x_T = [\texttt{[M]}^L]$, tại mỗi bước tính độ tự tin $c_i = \max_v p(v \mid x_t)$, chỉ mở khóa các vị trí tin cậy nhất và giữ `[M]` ở các vị trí còn lại để tinh chỉnh tiếp.

Thuật toán lấy mẫu ngược Confidence-Based Remasking

```
@torch.no_grad()
def sample_unmasking(model, steps=4, prompt=None, device="cpu"):
    model.eval()
    xt = torch.full((1, SEQ_LEN), MASK_IDX, dtype=torch.long, device=
    device) if prompt is None else \
         torch.tensor([[word2idx.get(w, MASK_IDX) for w in prompt]],
    device=device)
    fixed_mask = (xt != MASK_IDX)
    for s in range(1, steps + 1):
```

```
        t_val = 1.0 - (s - 1) / steps
        probs = F.softmax(model(xt, torch.tensor([t_val], device=device))[0], dim=-1)
        top_probs, top_preds = probs.max(dim=-1)
        num_remain = int(math.floor(SEQ_LEN * (1.0 - s / steps)))
        masked_idx = [i for i in range(SEQ_LEN) if xt[0, i] == MASK_IDX and not fixed_mask[0, i]]
        masked_idx.sort(key=lambda i: top_probs[i].item(), reverse=True)
        for idx in masked_idx[:max(0, len(masked_idx) - num_remain)]:
            xt[0, idx] = top_preds[idx]
    return xt
```

Kết quả khử mặt nạ từng bước của Mini-DLM ($T = 4$)

```
Bước 0 (t=1.00): [M] [M] [M] [M] [M] [M] [M] [M]
Bước 1 (t=0.75): [M] đi [M] [M] đến [M] [M] [M]
Bước 2 (t=0.50): [M] đi xe máy đến [M] [M] [M]
Bước 3 (t=0.25): [M] đi xe máy đến công_sở làm [M]
Bước 4 (t=0.00): hải đi xe máy đến công_sở làm việc
```

Quan sát Bảng 11, mô hình mở khóa mỏ neo ngữ pháp `đi`, `đến` (1,0000) trước, tiếp theo giải mã phương tiện `xe máy` (0,8504), thu hẹp ngữ cảnh thành `công_sở làm` (0,6992), và cuối cùng chốt `hải` cùng `việc` (0,9998).

Bước	Mức t	Số [M]	Vị trí mở	Độ tự tin	Trạng thái chuỗi trung gian x_t
0	$t = 1{,}00$	8	–	–	`[M] [M] [M] [M] [M] [M] [M] [M]`
1	$t = 0{,}75$	6	2, 5	1,0000	`[M] đi [M] [M] đến [M] [M] [M]`
2	$t = 0{,}50$	4	3, 4	0,8504	`[M] đi xe máy đến [M] [M] [M]`
3	$t = 0{,}25$	2	6, 7	0,6992	`[M] đi xe máy đến công_sở làm [M]`
4	$t = 0{,}00$	0	1, 8	0,9998	`hải đi xe máy đến công_sở làm việc`

Bảng 11: Quỹ đạo khử mặt nạ lặp từng bước theo độ tự tin (Confidence-Based Remasking).

Trong thử nghiệm **điền khuyết hai đầu (Bidirectional Infilling)** với chuỗi prompt:

$$\mathbf{x}_{\text{prompt}} = [\texttt{lan}, \texttt{đi}, \texttt{[M]}, \texttt{[M]}, \texttt{đến}, \texttt{thư_viện}, \texttt{[M]}, \texttt{[M]}]$$

qua $T = 3$ bước (Bảng 12), dựa vào hậu tố `thư_viện`, mô hình suy luận ngay hành động `đọc sách` (0,9961) trước khi điền phương tiện `xe buýt` (0,9993). Trên Google Colab (**T4 GPU**), chỉ cần đổi `QUICK_RUN = False` để tự động kích hoạt huấn luyện hỗn hợp FP16 (AMP) tốc độ cao.

Bước	Mức t	Số [M]	Vị trí mở	Độ tự tin	Trạng thái chuỗi đang điền khuyết
0–1	1,00 → 0,67	4	–	0,6024	`lan đi [M] [M] đến thư_viện [M] [M]`
2	$t = 0{,}33$	2	7, 8	0,9961	`lan đi [M] [M] đến thư_viện` **`đọc sách`**
3	$t = 0{,}00$	0	3, 4	0,9993	`lan đi` **`xe buýt`** `đến thư_viện đọc sách`

Bảng 12: Quỹ đạo điền khuyết hai đầu (Infilling) phối hợp ngữ nghĩa giữa tiền tố và hậu tố.

Tóm lại, thực nghiệm Mini-DLM đã minh họa trực quan cơ chế huấn luyện và quỹ đạo khử mặt nạ hai chiều trên PyTorch. Phần IX. sẽ tổng hợp và đối chiếu toàn diện giữa mô hình tự hồi quy (AR) và mô hình ngôn ngữ khuếch tán (DLM).

IX. So sánh tổng hợp AR và DLM: Đánh giá toàn diện và triển vọng tương lai

Sau khi khảo sát toàn diện từ lý thuyết xích Markov rời rạc đến thực nghiệm PyTorch, câu hỏi chiến lược đặt ra là: *Khi nào nên dùng mô hình tự hồi quy (AR), và khi nào mô hình khuếch tán ngôn ngữ (DLM) phát huy ưu thế vượt trội?* Mọi lựa chọn đều là sự đánh đổi giữa độ trễ, chất lượng sinh và chi phí phần cứng.

IX.1. Bảng ma trận so sánh đối đầu 10 tiêu chí kỹ thuật

Bảng 13 đối chiếu trực diện giữa Mô hình Tự hồi quy (*Autoregressive Models* – AR: GPT-4, LLaMA-3, Qwen-2.5) và Mô hình Khuếch tán Ngôn ngữ (*Diffusion Language Models* – DLM: LLaDA, Dream, Mercury) trên 10 tiêu chí kỹ thuật.

Tiêu chí đánh giá	Mô hình Tự hồi quy (AR Models)	Mô hình Khuếch tán Ngôn ngữ (DLM)
1. Hướng sinh token	Tuyến tính từ trái sang phải ($x_1 \to \cdots \to x_L$).	Đa hướng, song song, tinh chỉnh từ thô đến tinh.
2. Cơ chế chú ý	Mặt nạ nhân quả tam giác (Causal Mask).	Chú ý hai chiều toàn cục (Full Bidirectional Attention).
3. Độ trễ (*Latency*)	Tuyến tính với độ dài đầu ra: $\mathcal{O}(L)$ lượt chuyển tiếp.	Phụ thuộc số bước lấy mẫu S: $\mathcal{O}(S)$ lượt ($S \ll L$).
4. Thông lượng phục vụ	Bị nghẽn băng thông bộ nhớ GPU khi giải mã tuần tự.	Rất cao ở batch lớn nhờ giải mã song song đồng thời.
5. Động học KV Cache	Tích lũy đơn điệu; tối ưu dễ với PagedAttention.	Vô hiệu hóa ở dạng thuần; cần Block Diffusion / Fast-dLLM.
6. Infilling & Sửa	Rất kém; đòi hỏi biến đổi dữ liệu phức tạp (FIM).	Tự nhiên: cố định token đã biết và che vùng cần điền.
7. Lời nguyền đảo ngược	Bị ảnh hưởng nặng do học biểu diễn một chiều ($A \to B$).	Miễn nhiễm tự nhiên nhờ tiếp xúc ngữ cảnh hai chiều.
8. Hiệu quả 1 epoch	Cực cao; 100% token đóng góp vào hàm mất mát.	Kém hơn ở 1 epoch (chỉ tính loss trên token bị che).
9. Căn chỉnh & RLHF	Trưởng thành cao độ: PPO, DPO, GRPO trên từng bước.	Đang định hình: VRPO (LLaDA 1.5), Coupled-GRPO (DiffuCoder).
10. Hệ sinh thái	Rất cao: vLLM, TensorRT-LLM, SGLang, kernel tối ưu.	Đang phát triển tích cực trên runtime chuyên biệt.

Bảng 13: Ma trận so sánh 10 tiêu chí kỹ thuật giữa mô hình tự hồi quy (AR) và mô hình ngôn ngữ khuếch tán (DLM).

IX.2. Động học KV Cache và Những giới hạn thẳng thắn của DLM

Về kinh tế học suy luận, AR bị giới hạn bởi băng thông bộ nhớ (*Memory-Bound*, arithmetic intensity ≈ 1 FLOP/Byte do nạp trọng số để sinh từng token), trong khi DLM dịch chuyển sang giới hạn tính toán (*Compute-Bound*) nhờ xử lý song song L vị trí trên Tensor Cores, kết hợp Block Diffusion [9] và Fast-dLLM [10] để cắt giảm $> 93\%$ FLOPs chú ý. Tuy nhiên, DLM hiện vẫn chưa thể thay thế hoàn toàn AR ở bốn điểm hạn chế:

1. **Hiệu quả dữ liệu 1 epoch:** Ở 1 epoch, AR học trên 100% token trong khi DLM chỉ tính loss trên các vị trí bị che (trung bình 50%), đòi hỏi nhiều dữ liệu huấn luyện hơn.
2. **Đánh đổi khi giảm bước lấy mẫu ($S < 8$):** Giảm S quá thấp làm tăng Perplexity và phát sinh lỗi ghép đa mode (chimeric errors).
3. **Cơ chế dừng tự nhiên:** DLM toàn cục cần định trước chiều dài L hoặc kết hợp phân khối Block Diffusion để dừng linh hoạt theo `[EOS]`.
4. **Hệ sinh thái công nghiệp:** Hạ tầng phục vụ AR (vLLM, TensorRT-LLM) vẫn hoàn thiện hơn so với các thư viện DLM mới nổi.

IX.3. Tổng hợp tiến trình phát triển và Triển vọng nghiên cứu tương lai

Tiến trình tiến hóa của DLM kết tinh qua 5 cột mốc: (1) **D3PM** [3] (xích Markov rời rạc Q_t và ELBO); (2) **SEDD** [4] (CTMC liên tục, Concrete Score $s_t(y, x) = p_t(y)/p_t(x)$ và Score Entropy); (3) **MDLM** [5] & **MD4** [6] (trạng thái hấp thụ `[M]`, rút gọn ELBO về cross-entropy trọng số $1/t$); (4) **LLaDA** [2] & **Dream** [7] (mở rộng quy mô 7B–8B, giải Reversal Curse, chuyển giao trọng số từ AR); và (5) **Fast-dLLM** [10] (lưu đệm KV xấp xỉ và giải mã song song theo độ tự tin). Các thách thức mở và hướng nghiên cứu tương lai tập trung vào:

- **Luật co giãn quy mô (Scaling Laws) & Kiến trúc lai AR-DLM:** Xác lập tương quan định lượng giữa tham số, dữ liệu, số bước S và kết hợp lập kế hoạch vĩ mô (AR) với sinh song song (DLM).
- **Căn chỉnh RLHF & Kernel phần cứng:** Phát triển thuật toán hậu huấn luyện (VRPO, Coupled-GRPO) [8, 17] cùng custom Triton kernels cho attention hai chiều thưa.

Tóm lại, Diffusion Language Models không chỉ giải bài toán độ trễ suy luận mà còn mở ra bước chuyển tư duy từ chuỗi nhân quả một chiều sang quá trình tinh chỉnh thông tin hai chiều toàn diện, khẳng định vị thế trụ cột kiến trúc của trí tuệ nhân tạo tạo sinh thế hệ mới.

Tài liệu tham khảo

[1] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008. address: https://arxiv.org/abs/1706.03762.

[2] S. Nie et al., "Large Language Diffusion Models," *arXiv preprint arXiv:2502.09992*, 2025. address: https://arxiv.org/abs/2502.09992.

[3] J. Austin et al., "Structured Denoising Diffusion Models in Discrete State-Spaces," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 17 981–17 993. address: https://arxiv.org/abs/2107.03006.

[4] A. Lou et al., "Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution," in *International Conference on Machine Learning (ICML)*, Best Paper Award, 2024. address: https://arxiv.org/abs/2310.16834.

[5] S. S. Sahoo et al., "Simple and Effective Masked Diffusion Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. address: https://arxiv.org/abs/2406.07524.

[6] J. Shi et al., "Simplified and Generalized Masked Diffusion for Discrete Data," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. address: https://arxiv.org/abs/2406.04329.

[7] J. Ye et al., "Dream 7B: Diffusion Large Language Models," *arXiv preprint arXiv:2508.15487*, 2025. address: https://arxiv.org/abs/2508.15487.

[8] S. Gong et al., "DiffuCoder: Understanding and Improving Masked Diffusion Models for Code Generation," *arXiv preprint arXiv:2506.20639*, 2025. address: https://arxiv.org/abs/2506.20639.

[9] M. Arriola et al., "Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models," in *International Conference on Learning Representations (ICLR)*, 2025. address: https://arxiv.org/abs/2503.09573.

[10] C. Wu et al., "Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding," in *International Conference on Learning Representations (ICLR)*, 2026. address: https://arxiv.org/abs/2505.22618.

[11] Inception Labs. "Mercury: Commercial Diffusion Large Language Models with Sub-second Latency." (2025), address: https://inceptionlabs.ai (visited on 10/19/2026).

[12] J. Sohl-Dickstein et al., "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 2256–2265. address: https://arxiv.org/abs/1503.03585.

[13] J. Ho et al., "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851. address: https://arxiv.org/abs/2006.11239.

[14] X. L. Li et al., "Diffusion-LM Improves Controllable Text Generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 4328–4343. address: https://arxiv.org/abs/2205.14217.

[15] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186. address: https://arxiv.org/abs/1810.04805.

[16] J. Ou et al., "Your Absorbing Discrete Diffusion Secretly Models the Conditional Distributions of Clean Data," in *International Conference on Learning Representations (ICLR)*, 2025. address: https://arxiv.org/abs/2406.03736.

[17] F. Zhu et al., "LLaDA 1.5: Variance-Reduced Preference Optimization for Large Language Diffusion Models," *arXiv preprint arXiv:2505.19223*, 2025. address: https://arxiv.org/abs/2505.19223.

Phụ lục

Tài nguyên mã nguồn mở, kho lưu trữ và thực nghiệm

Để hỗ trợ học viên và các nhóm nghiên cứu dễ dàng tái lập kết quả, kiểm tra chéo các công thức toán học và tiếp cận trực tiếp với các codebase hàng đầu thế giới, dưới đây là danh mục tổng hợp các tài nguyên mã nguồn mở chính thức và công cụ đồng hành cùng chuyên đề:

- **Kho mã nguồn SEDD (Score Entropy Discrete Diffusion):**
  Kho lưu trữ chính thức của nhóm tác giả Aaron Lou, Chenlin Meng, Stefano Ermon (Đại học Stanford, ICML 2024 Best Paper):
  https://github.com/aaronlou/sedd
- **Kho mã nguồn MDLM (Masked Diffusion Language Models):**
  Kho lưu trữ chính thức của nhóm nghiên cứu Kuleshov Group (Đại học Cornell, NeurIPS 2024):
  https://github.com/kuleshov-group/mdlm
- **Mô hình nền tảng LLaDA 8B & LLaDA 1.5:**
  Kho lưu trữ mã nguồn, trọng số mở trên Hugging Face và tài liệu kỹ thuật của nhóm nghiên cứu ML-GSAI:
  https://github.com/ML-GSAI/LLaDA
- **Thư viện tăng tốc suy luận Fast-dLLM (NVIDIA Research):**
  Giải pháp tăng tốc giải mã song song và approximate KV Cache cho Diffusion LLM (ICLR 2026):
  https://github.com/NVlabs/Fast-dLLM
- **Mô hình Dream 7B (HKU NLP & Huawei Noah's Ark Lab):**
  Triển khai chuyển giao trọng số từ mô hình tự hồi quy sang mô hình khuếch tán:
  https://github.com/HKUNLP/Dream
- **Sổ tay thực hành Mini-DLM (AIO2026 Companion Notebook):**
  Mã nguồn PyTorch thuần huấn luyện mô hình khuếch tán mini trên CPU và Colab T4, kèm trực quan hóa quỹ đạo unmasking:
  Đường dẫn tệp nội bộ: `case_study/mini_dlm_case_study.ipynb`
- **Hệ thống kiểm toán số liệu tính tay (*Numerical Verification Suite*):**
  Script Python độc lập xác thực toàn bộ các giá trị phân phối xác suất và ma trận:
  Đường dẫn tệp nội bộ: `scripts/verify_hand_calculations.py`

- Hết -